

Rapport Mini-projet : Conception et développement d'une base de données - PARTIE 2

Module : TI503N – Bases de données 1



Réalisé par : Mohammed EL KARCHAL

05/10/2025

Sommaire

III : Modèles logique & physique

III.A — Modèle Logique des Données (MLD) :

III.B — Modèle Physique des Données (MPD)

IV. : Contraintes et intégrité des données

V : Insertion des données

V.A — Données choisies

VI : Interrogation (requêtes SQL)

VI.A — Scénario d'utilisation

VI.B — Requêtes essentielles

VII : Conclusion

III : Modèles logique & physique

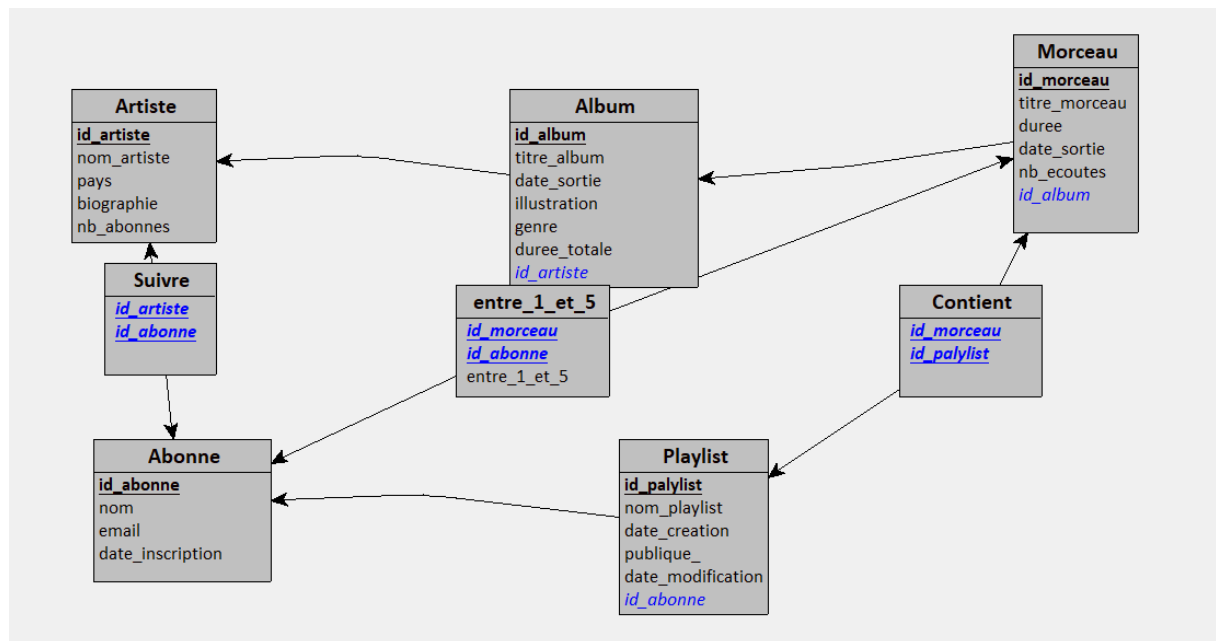
Dans cette deuxième partie, je passe de la modélisation (MCD/MLD) à l'implémentation réelle dans MySQL. L'objectif est de créer physiquement les tables, d'insérer des données réalistes et d'interroger la base afin de valider le modèle et préparer l'exploitation future.

III.A — Modèle Logique des Données (MLD) :

Le MLD a été généré automatiquement dans *Looping* à partir du MCD. Il traduit les entités en tables relationnelles avec clés primaires et étrangères, en respectant la 3e forme normale (3FN).

Les tables obtenues sont : Artiste, Abonne, Album, Morceau, Playlist, Suivre, Evaluation, Contient.

Les associations *Publie*, *Compose*, *Note*, *Contient* ont été transformées en clés étrangères et, si nécessaire, en tables d'association.



III.B — Modèle Physique des Données (MPD)

Le **MPD** correspond au schéma SQL exécuté dans MySQL Workbench. Il contient la définition des tables, les types de données, les **PRIMARY KEY**, **FOREIGN KEY** et les contraintes de validité.

Le script complet est dans le fichier **1_creation.sql** de mon dépôt GitHub. Ci-dessous un extrait représentatif :

```
1 • CREATE DATABASE IF NOT EXISTS plateforme_musicale;
2 • USE plateforme_musicale;
3
4 • CREATE TABLE Artiste (
5     id_artiste INT PRIMARY KEY,
6     nom_artiste VARCHAR(100),
7     pays VARCHAR(50),
8     biographie TEXT,
9     nb_abonnes INT
10 );
11
12 • CREATE TABLE Abonne (
13     id_abonne INT PRIMARY KEY,
14     nom VARCHAR(100),
15     email VARCHAR(150),
16     date_inscription DATE
17 );
18
19 • CREATE TABLE Playlist (
20     id_playlist INT PRIMARY KEY,
21     nom_playlist VARCHAR(100),
22     date_creation DATE,
23     publique BOOLEAN NOT NULL,
24     date_modification DATE,
```

IV : Contraintes et intégrité des données

Afin de garantir l'intégrité et la cohérence de la base, plusieurs contraintes ont été ajoutées :

- **Contrainte d'unicité** sur l'email des abonnés (aucun doublon).
- **Contrainte CHECK** pour valider que les notes sont entre 1 et 5.
- **Contrainte CHECK** pour éviter les valeurs négatives (ex : nombres d'écoutes ou abonnés).
- **Création d'index** pour optimiser les performances lors des jointures.

Scénario : un abonné explore la plateforme pour découvrir les morceaux les plus écoutés, consulter les albums d'un artiste, voir ses playlists et les notes de la communauté. Les requêtes suivantes valident ces besoins.

VI.B — Requêtes essentielles

1) Morceaux les plus écoutés (TOP 5) :

```
30  /* -----  
31      3) Morceaux les plus écoutés (TOP 5)  
32  ----- */  
33  • SELECT m.titre_morceau, m.nb_ecoutes, al.titre_album, ar.nom_artiste  
34      FROM Morceau m  
35      JOIN Album al ON al.id_album = m.id_album  
36      JOIN Artiste ar ON ar.id_artiste = al.id_artiste  
37      ORDER BY m.nb_ecoutes DESC  
38      LIMIT 5;
```

Ou on peut remarquer que les morceaux de **Damso** dominent en nombre d'écoutes.**2)**

Moyenne des notes par morceau :

```
43  • SELECT m.titre_morceau, AVG(e.note) AS note_moyenne, COUNT(*) AS nb_notes  
44      FROM Morceau m  
45      JOIN Evaluation e ON e.id_morceau = m.id_morceau  
46      GROUP BY m.id_morceau  
47      ORDER BY note_moyenne DESC, nb_notes DESC;
```

Les titres les mieux notés ont une moyenne entre 4 et 5.

3) Playlists publiques avec nombre de morceaux

```
52  • SELECT p.id_playlist, p.nom_playlist, p.date_creation, COUNT(c.id_morceau) AS nb_morceaux  
53      FROM Playlist p  
54      LEFT JOIN Contient c ON c.id_playlist = p.id_playlist  
55      WHERE p.publique = TRUE  
56      GROUP BY p.id_playlist  
57      ORDER BY nb_morceaux DESC, p.date_creation DESC;
```

Rap & Chill est fournie et visible par tous.

4) Abonnés qui suivent un artiste

```
74  • SELECT ar.nom_artiste, ab.id_abonne, ab.nom, ab.email  
75      FROM Suivre s  
76      JOIN Artiste ar ON ar.id_artiste = s.id_artiste  
77      JOIN Abonne ab ON ab.id_abonne = s.id_abonne  
78      WHERE ar.nom_artiste = @nom_artiste  
79      ORDER BY ab.nom;
```

Ça permet d'identifier la base de fans par artiste.

VII.Conclusion

La base plateforme musicale a été implémentée avec succès dans MySQL. Les scripts de création, insertion et interrogation fonctionnent et valident la cohérence du modèle.

Les vues facilitent les analyses.

Cette partie confirme que le schéma est prêt pour une exploitation applicative